

Interview Questions: Fraud Network Detection Project

25 questions, drawn directly from the project, with model answers for interview prep

These questions mirror how an interviewer would probe a project like this: starting with fundamentals, moving into your specific technical decisions, and ending with the kind of judgment/depth questions that separate a memorized project from genuine understanding. Use the model answers as a starting point, then practice explaining them in your own words with your own numbers.

Section 1: Project Fundamentals

Q1. What problem were you solving in this project, and why is it hard?

EASY

A: Detecting fraudulent credit card transactions in real time from a dataset of 1.85 million simulated transactions across 1,000 customers and 800 merchants. It's hard mainly because of extreme class imbalance - only 0.58% of transactions are fraud - so a naive model can score 99.4% accuracy while catching zero fraud. The challenge is building a model that finds the rare signal without drowning in false positives.

Q2. Why did you call this a 'network' detection engine rather than just a classifier?

EASY

A: Beyond treating each transaction as an independent row, I modeled customers and merchants as nodes in a graph connected by transaction edges, using NetworkX. This let me extract relational features - like how many distinct merchants a customer uses, or how many distinct customers a merchant serves - and also visualize fraud connections as a subgraph to spot clustering patterns.

Q3. Walk me through your overall pipeline from raw data to final model.

EASY

A: Load and clean the data, exploratory analysis (fraud rate by hour/amount/category), build a customer-merchant graph, engineer time-aware and behavioral features, split into train/test (already time-split in the source data), train and compare Logistic Regression, Random Forest, and XGBoost using AUC-PR as the primary metric, tune the decision threshold, then finalize with feature importance and a network visualization.

Q4. Why was accuracy the wrong metric to optimize here?

EASY

A: With 99.4% of transactions legitimate, a model predicting 'not fraud' every single time scores 99.4% accuracy while being completely useless. I used precision, recall, F1-score, ROC-AUC, and especially AUC-PR (area under the precision-recall curve), which is far more informative for rare-event problems since it focuses on how well the model ranks and separates the minority class.

Q5. What was your final model and what were its results?

EASY

A: Random Forest with `class_weight='balanced'`, operating at a tuned decision threshold of 0.2. Final results: 63% precision, 67% recall, F1-score of 0.65 on the fraud class, with ROC-AUC of 0.959 and AUC-PR of 0.693.

Section 2: Feature Engineering

Q6. What was your single most important engineered feature, and why did it work?

MEDIUM

A: `amt_zscore` - the number of standard deviations a transaction's amount is from that specific customer's own historical average. Legitimate transactions averaged a z-score near 0, while fraud transactions averaged 3.2 (median 2.26). It worked because it captures relative, personalized anomaly rather than absolute spending, which is more robust across customers with very different normal spending habits.

Q7. You tested geographic distance between customer and merchant as a feature. Why didn't it work, and what did you learn from that?

MEDIUM

A: I computed Haversine distance between customer home coordinates and merchant coordinates, expecting fraud to correlate with implausible distances. The distributions were nearly identical for fraud and legitimate transactions (~76 km mean for both) - a known artifact of how the Sparkov simulator places merchant coordinates near the customer regardless of fraud status. The lesson: not every intuitive feature pans out, and testing it (rather than assuming) is real feature-engineering discipline; it also taught me to sanity-check a simulated dataset's generation assumptions before trusting a feature.

Q8. What real-time / velocity feature did you build, and what does it capture?

MEDIUM

A: `trans_count_last_1h` - a rolling count, per customer, of how many transactions occurred in the preceding hour, computed with a `groupby + rolling('1h')` on a time-indexed series. It captures transaction bursts, a classic fraud signal (e.g. a stolen card used rapidly). Fraud transactions averaged 1.68 per hour versus 1.19 for legitimate ones - a real but moderate signal in this dataset.

Q9. How did you turn your customer-merchant graph into features a model could use?

MEDIUM

A: By computing node degree from each side of the bipartite graph: `customer_merchant_diversity` (how many distinct merchants a customer has transacted with) and `merchant_customer_diversity` (how many distinct customers a merchant serves), both via `groupby().nunique()`. I also computed `merchant_fraud_rate` as a node-level aggregate label statistic.

Q10. Why did you convert `cc_num` and `zip` to strings instead of leaving them as integers?

EASY

A: Because they're identifiers, not quantities - there's no meaningful numeric relationship between two credit card numbers or zip codes (a higher `cc_num` doesn't mean anything). Leaving them as `int64` risks a model treating them as ordinal/continuous, so I cast them to `string/categorical` to signal their true nature.

Section 3: Data Leakage & Debugging (Deep Dive)

Q11. You mentioned discovering a bug that changed your results dramatically. Walk me through it.

HARD

A: My tree-based models (Random Forest, XGBoost) initially scored far worse than plain Logistic Regression on AUC-PR (0.04-0.06 vs. 0.15-0.23), which is backwards from what's normally expected on tabular fraud data. I suspected distribution shift and compared feature means between train and test - `customer_txn_count` averaged 1,819 in train but only 781 in test, a huge, suspicious gap.

Q12. What was actually causing that gap?

HARD

A: I had computed `customer_txn_count` and similar aggregates separately on the train and test CSV files using a plain `groupby().count()`. Since the test file covers a shorter, later time period for the same customers, every customer looked artificially 'new' with far fewer transactions in test - purely a counting artifact of the time window, not a real behavior change.

Q13. Why did this hurt tree-based models more than Logistic Regression?

HARD

A: Tree models split hard on threshold values (e.g. "if `txn_count` > 1000, less likely fraud"), so an artificial, systematic shift in that value between train and test badly confuses their learned splits. Logistic Regression applies one smooth linear weight across the whole range, making it comparatively robust to this kind of shift, even though the underlying feature was still flawed for both models.

Q14. How did you fix it?

HARD

A: I concatenated train and test into a single chronologically sorted timeline (tagging which rows originally belonged to which set), recomputed every aggregate as a cumulative, expanding-window calculation using `cumcount()` and `expanding().mean().shift(1)`, then split the data back into train/test using the saved tag. This way, every feature reflects only information available strictly before that transaction, mirroring a real production system.

Q15. What was the measured impact of that fix?

HARD

A: Random Forest's AUC-PR rose from 0.043 to 0.693 - over a 10x improvement - and XGBoost went from 0.060 to 0.632, using the exact same algorithms and hyperparameters. It proved conclusively that the original poor performance was a data engineering problem, not a modeling problem.

Q16. What is data leakage in general, and can you give another example from your project?

MEDIUM

A: Leakage is when information that wouldn't actually be available at prediction time ends up inside a feature, inflating test performance in a way that won't hold up in production. Another example I handled carefully: `merchant_fraud_rate` uses the `is_fraud` label itself, so I made sure it was computed only from past transactions (`expanding().shift(1)`) relative to each row, never including the current or future transaction's own label.

Q17. Why is a time-based train/test split more realistic here than a random split, and what extra care does it require?

HARD

A: In production, you always predict on future transactions using only past data - a random split would let the model implicitly learn from 'future' rows during training, which can't happen in real deployment. The extra care it requires is exactly what caused the bug: every feature must be computed causally (using only past data relative to each row), or the time-based split introduces its own artifacts like the one I found.

Section 4: Modeling & Evaluation

Q18. Why did you choose AUC-PR over ROC-AUC as your primary metric?

MEDIUM

A: ROC-AUC can look deceptively good on highly imbalanced data because it's influenced by the large number of true negatives, which are trivial to get right when 99.4% of the data is negative. AUC-PR focuses specifically on precision and recall for the positive (fraud) class, making it a much more honest measure of how useful the model actually is for catching rare events.

Q19. How did `class_weight='balanced'` and `scale_pos_weight` work in your models, and did more weighting always help?

MEDIUM

A: Both mechanisms upweight the rare class during training so misclassifying fraud is penalized more heavily than misclassifying a legitimate transaction. Interestingly, more aggressive weighting did NOT always help in my testing - I swept `scale_pos_weight` from 1 to 171.75 for XGBoost and found performance varied non-monotonically, so I had to test empirically rather than assume 'more correction is better'.

Q20. Why did you tune the decision threshold instead of using the default 0.5?

MEDIUM

A: A classifier's raw output is a probability, and 0.5 is an arbitrary default that's rarely optimal for imbalanced problems. I plotted precision and recall across a range of thresholds and picked 0.2 as a practical operating point - it catches about two-thirds of fraud while keeping roughly 63% precision, a workable balance for a human review queue, rather than accepting the default's very different tradeoff.

Q21. Compare Logistic Regression, Random Forest, and XGBoost - when would you choose each?

MEDIUM

A: Logistic Regression is fast, interpretable, and robust to noisy features but limited to linear relationships - good as a baseline or when interpretability matters most. Random Forest captures non-linear interactions via an ensemble of trees averaged together, and was my best performer here on AUC-PR. XGBoost builds trees sequentially to correct prior errors and had the best ROC-AUC; it's usually the strongest choice on structured/tabular data in production settings, though it can be more sensitive to hyperparameter tuning.

Q22. Your feature importance showed 'amt' as the top feature by far. Doesn't that make the model too simple?

MEDIUM

A: It's a fair concern, but the importance profile improved substantially after adding `amt_zscore` and `is_night` - together they made up over 25% of importance, alongside meaningful (though smaller) contributions from the network features and velocity feature. A model leaning on a few strong signals isn't inherently bad; what matters is validating those signals are genuine (which the EDA confirmed) rather than leaked or spurious.

Q23. If you were told this model has to run in true real-time production (scoring transactions the instant they happen), what would you change?

HARD

A: The main change is ensuring every feature is computed incrementally and causally at scoring time - e.g. maintaining a running per-customer average/std and per-merchant fraud rate in a fast key-value store (like Redis) updated after each transaction, rather than recomputing over the full historical dataset. I'd also cap memory/latency by using bounded rolling windows instead of full-history expanding windows, and add monitoring for feature drift over time.

Section 5: Reflection & Judgment

Q24. What was the most valuable lesson from this project, and how would you apply it to a new problem?

MEDIUM

A: That how features are computed matters more than which algorithm you choose - the biggest single improvement in the whole project (AUC-PR going from 0.043 to 0.693) came from fixing a feature engineering bug, not from switching models. On a new problem, my first instinct now is to sanity-check feature distributions between train and test before trusting any model comparison, especially with time-ordered data.

Q25. If you had more time or resources, what would you improve next?

MEDIUM

A: Three things: first, build an actual streaming simulation that replays transactions in time order and scores them live, to validate the causal features truly behave correctly outside of batch evaluation. Second, experiment with a graph neural network (e.g. GraphSAGE) that learns directly from the customer-merchant graph structure instead of relying only on hand-engineered degree statistics. Third, add automated community detection (e.g. Louvain) on the graph to surface fraud rings or clusters algorithmically rather than sampling fraud cases manually for visualization.